

Reviewer Pack — Permutation-Orbit Stability of Tropical Doubling Defect at β ...

Entry 6ecf7fd15eef · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 10:02:30 UTC

Permutation-Orbit Stability of Tropical Doubling Defect at $\beta=5$

Entry ID: 6ecf7fd15eef **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-29 10:02:30 UTC

1. Conjecture

Field A (mathematical branch): Min-plus algebra (S_n permutation symmetries of min-plus self-convolution operators on Z_n) **Field B** (complexity object): Tropical Fourier analysis (Maslov-dequantized doubling defect $\Delta(f, \beta, n) := |\text{MFC}(g) - 2 \cdot \text{MFC}(f)|$, g = tropical self-convolution of f)

Statement:

Fix $\beta=5$. For a tropical polynomial $f: Z_n \rightarrow R$ let $O(f) := \{f \circ \sigma : \sigma \in S_n\}$ be its permutation orbit (every $f' \in O(f)$ has identical multiset of values, hence identical $\text{disc}(f') = \max f - \min f$, while min-plus self-convolution $g(x) = \min_y [f(y) + f(x-y)]$ is NOT a permutation invariant). Conjecture (one inferential step toward SC4 ' Δ depends only on $\text{disc}(f')$ '): for every $n \in \{8, 12, 16, 20, 24, 32, 40\}$ and every base polynomial f drawn i.i.d. Uniform $[0, 1]$ under a seed $s \in \{1, \dots, 30\}$, the orbit coefficient-of-variation $\text{CV}(f) := \text{std}_{\sigma} [\Delta(f \circ \sigma, 5, n)] / \max(\text{mean}_{\sigma} [\Delta(f \circ \sigma, 5, n)], 10^{-6})$ estimated over 50 uniform random $\sigma \in S_n$ satisfies $\text{CV}(f) \leq 0.25$; a single (n, s) pair with $\text{CV}(f) > 0.25$ falsifies it.

Rationale (proposer's reasoning):

SC4 asserts $\text{disc}(f)$ is a sufficient statistic for Δ . Permutation orbits exactly hold the disc fixed while varying every other shape feature, so SC4 logically forces $\text{CV} \rightarrow 0$ on orbits. Because tropical self-convolution g is spatially sensitive (it mixes pairs $(y, x-y)$), invariance of $\text{MFC}(g) - 2 \cdot \text{MFC}(f)$ under reshuffling of f would be the first non-trivial structural witness that only the value spread, not the arrangement, drives the doubling defect — and a single high-CV orbit cleanly falsifies SC4 without any asymptotic guesswork.

Taxonomy category: tropical_discrepancy_finite_beta (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 5507e1cd018d233d

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across all $7 \times 30 = 210$ (n,s) pairs with $n \in \{8, 12, 16, 20, 24, 32, 40\}$ and seed $s \in \{1..30\}$, the orbit $CV(f) = \text{std_}\sigma[\Delta] / \max(\text{mean_}\sigma[\Delta], 1e-6)$ estimated over 50 uniform random $\sigma \in S_n$ must satisfy $CV(f) \leq 0.25$ for every pair; one pair with $CV > 0.25$ falsifies.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.97	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.97	SAFE	SAFE
ALGEBRIZATION	SAFE	0.96	SAFE	SAFE
KARP_LIPTON	SAFE	0.98	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - tropical polynomial min-plus convolution permutation invariance - Maslov dequantization Fourier tropical doubling defect-min-plus self-convolution symmetric group orbit stability

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(A):
    m, n = len(A), len(A[0])
    for i in range(m):
        max_row = i + max(range(i, m), key=lambda r: abs(A[r][i]))
        A[i], A[max_row] = A[max_row], A[i]
        for j in range(n):
            if i == j:
                A[i][j] = Fraction(1, A[i][j])
            else:
                A[i][j] = Fraction(0)
        for k in range(m):
            if k != i:
                factor = A[k][i]
                for j in range(n):
                    A[k][j] -= factor * A[i][j]

def matrix_multiply(A, B):
    m, n, p = len(A), len(B), len(B[0])
    C = [[Fraction(0) for _ in range(p)] for _ in range(m)]
    for i in range(m):
        for j in range(n):
```

```

        for k in range(p):
            C[i][k] += A[i][j] * B[j][k]
    return C

def inverse(A):
    m, n = len(A), len(A[0])
    if m != n:
        raise ValueError("Matrix must be square")
    I = [[Fraction(1) if i == j else Fraction(0) for j in range(n)] for i in range(m)]
    A_augmented = [row + col for row, col in zip(A, I)]
    gaussian_elimination(A_augmented)
    return [row[n:] for row in A_augmented]

def maslov_tft(f, beta, n):
    h = [f[(x - y) % n] + f[y] for x in range(n) for y in range(n)]
    k_values = list(range(1, n))
    exp_sum = sum(math.exp(-beta * h[x]) * math.cos(2 * math.pi * 1j * k * x / n) for k in k_values)
    return abs(exp_sum)

def cv(f, n):
    g = [min(f[(x - y) % n] + f[y] for y in range(n)) for x in range(n)]
    delta = [maslov_tft(g, 5, n) - 2 * maslov_tft(f, 5, n) for _ in range(50)]
    mean_delta = sum(delta) / len(delta)
    std_delta = math.sqrt(sum((x - mean_delta) ** 2 for x in delta) / len(delta))
    return std_delta / max(mean_delta, 1e-6)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [8, 12, 16, 20, 24, 32, 40]
    results = []
    for n in n_values:
        f = [random.random() for _ in range(n)]
        deltas = [cv(f, n) for _ in range(50)]
        cv_value = sum(deltas) / len(deltas)
        results.append({
            "n": n,
            "cv": cv_value
        })
    mean_cv = sum(result["cv"] for result in results) / len(results)
    max_n = max(result["n"] for result in results)
    conjecture_holds = all(result["cv"] <= 0.25 for result in results)
    counterexample = "" if conjecture_holds else "CV > 0.25"
    return {
        "metric_name": "orbit_cv",
        "metric_value": mean_cv,
        "instances_tested": len(results),
        "n_max": max_n,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or list(range(2, 32))
    results = []
    for seed in seeds:

```

```

result = run_trial(seed)
print(f"TRIAL: {{\"seed\": {seed}, **{result}}}"))
results.append(result)

mean_cv = sum(result["metric_value"] for result in results) / len(results)
support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)
if all(result["conjecture_holds"] for result in results):
    print(f"RESULT: SUPPORTED mean={mean_cv} std=NA support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_cv} std=NA support_fraction={support_fraction}")
else:
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"CV > 0.25\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d28a228a.py", line 95, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d28a228a.py", line 71, in run_trial
    deltas = [cv(f, n) for _ in range(50)]
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d28a228a.py", line 60, in cv
    delta = [maslov_tft(g, 5, n) - 2 * maslov_tft(f, 5, n) for _ in range(50)]
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d28a228a.py", line 55, in maslov_tft
    exp_sum = sum(math.exp(-beta * h[x]) * math.cos(2 * math.pi * 1j * k * x / n) for k in k_values)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d28a228a.py", line 55, in <genexpr>
    exp_sum = sum(math.exp(-beta * h[x]) * math.cos(2 * math.pi * 1j * k * x / n) for k in k_values)
                  ^
NameError: name 'x' is not defined

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a NameError ('x' is not defined' in a generator expression) before any CV values were computed, so neither the support nor the falsification condition can be evaluated. | next: Fix the maslov_tft implementation (bind x in the generator, e.g., sum over x in range(n) with proper real-valued Fourier kernel) and rerun the 210 (n,s) pairs.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	47120
2	propose	claude_max	opus	0	0	51346
3	preregistration	claude_max	opus	0	0	5647
4	novelty	claude_max	opus	0	0	4615
5	novelty	claude_max	opus	0	0	6421
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16931
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14797
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14663
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13204
10	judge	claude_max	opus	0	0	9351

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 184094 ms total latency. Provider mix: {'claude_max': 6, 'ollama_remote': 4}

(full prompt+response transcripts available in `research/audit/6ecf7fd15eef.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/6ecf7fd15eef.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/6ecf7fd15eef.tar.gz` (if generated)